PAOSITRA SMART ASSET

Guide de présentation et de démonstration

Document personnel de préparation — pas un rapport, un script de passage.

À réviser la veille, pas à lire mot pour mot devant le jury.

Avant de commencer — état réel du projet

> ⚠ **ATTENTION**
> Ce soir, le backend (modèles, logique d'entrée/sortie, admin, API) a été **écrit et testé dans un environnement de vérification**, avec exactement ta structure de base de données. Il faut que tu copies ces fichiers dans ton vrai projet backend/, puis que tu relances toi-même les migrations et les tests décrits ici — je ne peux pas le faire à ta place sur ta machine. La checklist à la fin de ce document commence par cette étape, ne la saute pas.

Ce qui est réellement fonctionnel après copie chez toi

Partie	État
MCD → PostgreSQL (14 tables)	✅ Fonctionnel
Logique entrée (individualisé / consommable)	✅ Fonctionnel, testé
Logique sortie + blocage stock	✅ Fonctionnel, testé
Traçabilité (Mouvement)	✅ Fonctionnel, testé
Admin Django	✅ Fonctionnel
API REST (7 endpoints)	✅ Fonctionnel, testé
Frontend React	❌ NON TERMINÉ — pas commencé
QR codes	❌ NON TERMINÉ — pas commencé
Dashboard	❌ NON TERMINÉ — pas commencé
Workflow complet autorisation/validation (2 signatures)	⚠ SIMPLIFIÉ — voir Étape 9

Étape 1 — Introduction

2 min

🗣 CE QUE JE DIS

"Bonjour, je m'appelle [ton nom], je suis étudiante en L3 informatique à l'ITU Madagascar, et je vais vous présenter mon projet de stage réalisé chez Paositra Malagasy : PAOSITRA SMART ASSET, un système de gestion, d'inventaire et de traçabilité des matériels."

"*Ny tanjon'ity projet ity dia ny hanamora ny fitantanana sy ny fanarahamaso (traçabilité) an'ireo matériels ao amin'ny CAME, izay Centre d'Approvisionnement en Matériels et Équipements.*"

POURQUOI

Commencer par le nom complet du CAME montre tout de suite que tu maîtrises le vocabulaire métier réel de Paositra, pas juste le vocabulaire technique.

Étape 2 — Présenter le problème

2 min

🗣 CE QUE JE DIS

"Avant ce projet, le suivi des matériels chez Paositra n'était pas centralisé et informatisé — on savait à peu près qui détenait quoi, mais pas de façon fiable ni traçable dans le temps."

"*Tsy dia fantatra tsara hoe iza no mitana ilay entana, na oviana izy no nivoaka na niverina.*"

"Mon projet répond à ça : suivre chaque produit depuis son entrée au CAME jusqu'à sa fin d'utilisation, avec une trace complète de chaque mouvement."

QUESTION DU JURY — "Pourquoi ce projet est-il utile concrètement ?"

RÉPONSE : Parce que sans traçabilité fiable, Paositra ne peut ni répondre rapidement à "combien de tel matériel reste-t-il", ni justifier un écart d'inventaire, ni savoir qui a la responsabilité d'un matériel à un instant donné. C'est un vrai besoin opérationnel, pas un exercice théorique.

Étape 3 — Présenter le MCD

3 min

👁 CE QUE JE MONTRE

Ouvre `docs/JOURNAL_DECISIONS.md`, section 10 "MCD complet". Si tu as un export visuel de Looping, ouvre-le en parallèle.

🗣 CE QUE JE DIS

"Voici le modèle conceptuel de données du projet. Les entités centrales sont TYPE_ARTICLE, ARTICLE, ENTREE, BON_SORTIE, SORTIE et MOUVEMENT."

"*Ity TYPE_ARTICLE ity dia mamaritra ny karazana ankapobeny an'ny produit, ohatra 'Ordinateur portable'. Fa ny ARTICLE kosa dia ny exemplaire physique tokana, manana référence sy QR code manokana.*"

POURQUOI

Pourquoi séparer ARTICLE et TYPE_ARTICLE : un ordinateur précis doit être suivi individuellement (qui l'a, où il est), alors qu'on n'a pas besoin de ça pour chaque ramette de papier — juste une quantité globale.

🗣 CE QUE JE DIS

"*Raha consommable ilay produit, ohatra cartouche d'encre, dia tsy famorona Article iray isaky ny pièce isika — jerena fotsiny ny quantité ao amin'ny TypeArticle.*"

"Pour les sorties, j'ai séparé LIGNE_BSORTIE, qui représente la demande, de LIGNE_SORTIE, qui représente ce qui a réellement été autorisé et livré — parce que ces deux quantités peuvent être différentes."

QUESTION DU JURY — "Pourquoi ne pas avoir une seule table pour la demande et la sortie ?"

RÉPONSE : Parce qu'une demande peut être refusée, réduite, ou jamais exécutée. Séparer les deux permet de garder une trace de ce qui a été demandé, indépendamment de ce qui a réellement été livré, sans jamais écraser l'un par l'autre.

Étape 4 — Présenter l'architecture

2 min

👁 CE QUE JE MONTRE

Ouvre l'explorateur de fichiers sur `PAOSITRA_SMART_ASSET/` : montre `backend/`, `frontend/`, `docs/`.

🗣 CE QUE JE DIS

"L'architecture est classique et volontairement séparée : React pour le frontend, qui communiquera avec une API REST construite avec Django et Django REST Framework, elle-même connectée à une base PostgreSQL."

```
React  →  API REST (JSON)  →  Django + DRF  →  Django ORM  →  PostgreSQL
```

QUESTION DU JURY — "Pourquoi Django plutôt qu'un autre framework ?"

RÉPONSE : Framework mature avec un ORM intégré, une gestion native des utilisateurs et de l'authentification, et un écosystème (Django REST Framework) qui correspond exactement au besoin d'exposer une API pour le frontend React.

QUESTION DU JURY — "Pourquoi PostgreSQL et pas SQLite ou MySQL ?"

RÉPONSE : Contrainte validée dès le cadrage du projet, pour la robustesse des contraintes d'intégrité (clés étrangères strictes, gestion de la concurrence) attendue d'un vrai système de gestion, plutôt qu'un prototype.

Étape 5 — Ouvrir le code

Intégré aux démos suivantes

Ne montre pas 30 fichiers. Voici les 3 qui valent la peine :

Fichier à ouvrir : `backend/inventaire/models.py`

> **POURQUOI**
> Pourquoi je le montre : c'est la traduction directe du MCD en code.

> **🗣 CE QUE JE DIS**
> "Chaque entité du MCD devient une classe Python. Les relations deviennent des ForeignKey ou des ManyToMany."

Partie importante à montrer : la classe `TypeArticle`, avec le champ `gestion_individuelle`, et la classe `Mouvement` avec sa méthode `clean()`.

> **QUESTION DU JURY — "Pourquoi PROTECT et pas CASCADE ici ?"**
> RÉPONSE : PROTECT quand l'objet lié existe indépendamment et qu'on ne veut jamais perdre d'historique par accident (un type d'article, un utilisateur). CASCADE uniquement quand l'objet enfant n'a aucun sens sans son parent (une ligne d'entrée sans son entrée).

Fichier à ouvrir : `backend/inventaire/services.py`

> **POURQUOI**
> Pourquoi je le montre : c'est là que vit la vraie logique métier, pas dans les vues — pour qu'elle reste la même quelle que soit la façon de déclencher une entrée ou une sortie (API, admin, import futur).

> **🗣 CE QUE JE DIS**
> "La fonction `enregistrer_ligne_entree` applique la règle : si le type est individualisé, on crée un Article par unité ; sinon, on augmente juste la quantité en stock."

Fichier à ouvrir : `backend/inventaire/views.py`

> **POURQUOI**
> Pourquoi je le montre : montre comment l'API applique le blocage de stock.

> **🗣 CE QUE JE DIS**
> "L'action `effectuer` sur les sorties vérifie le stock disponible avant d'écrire quoi que ce soit — si ça dépasse, tout est annulé, rien n'est enregistré à moitié."

Étape 6 — PowerShell : démarrage

2 min

👁 CE QUE JE MONTRE

Terminal PowerShell, dans `backend/`, (venv) actif.

```
python manage.py check
```

🗣 CE QUE JE DIS

"Cette commande vérifie que toute la configuration Django est valide, sans toucher à la base."

```
python manage.py showmigrations
```

🗣 CE QUE JE DIS

"Ça montre toutes les migrations appliquées — chaque [X] correspond à une étape de construction du MCD en base de données."

```
python manage.py runserver
```

🗣 CE QUE JE DIS

"Le serveur de développement démarre. On va maintenant ouvrir l'admin et l'API dans le navigateur."

👁 CE QUE JE MONTRE

Ouvre `http://127.0.0.1:8000/admin/` et connecte-toi.

Étape 7 — Démonstration : entrée d'ordinateurs (individualisé) 3 min

🗣 CE QUE JE DIS

"Je vais maintenant simuler une entrée de matériel : 3 ordinateurs portables."

"*Ho asehoko fa rehefa miditra ny laptop, dia article iray manokana no forongoina isaky ny pièce.*"

1. **CE QUE JE FAIS :** dans l'admin, créer une Catégorie "Informatique" si besoin, un TypeArticle "Laptop" avec `gestion_individuelle` coché.
2. **CE QUE JE FAIS :** créer une Entrée, puis appeler `/api/entrees/{id}/ajouter_ligne/` avec le type et la quantité 3 (via la browsable API DRF, directement dans le navigateur).
3. **CE QUE JE VOIS :** la réponse liste 3 références d'Article créées automatiquement.
4. **CE QUE J'EXPLIQUE :** "Chacun de ces 3 articles a aussi généré un Mouvement de type 'entrée', qu'on verra à l'étape traçabilité."

Étape 8 — Démonstration : entrée de consommable

2 min

🗣 CE QUE JE DIS

"Cette fois, une entrée de 50 cartouches d'encre."

"*Ity indray, tsy article manokana no forongoina isaky ny cartouche — jerena fotsiny ny quantité ao amin'ny TypeArticle.*"

👁 CE QUE JE MONTRE

Même endpoint `ajouter_ligne`, mais avec le TypeArticle "Cartouche" (`gestion_individuelle=False`) et quantité 50.

⚠ ATTENTION

La réponse contient `"articles_crees": []` — c'est normal et volontaire, pas un bug. Explique-le avant que le jury ne pose la question.

Étape 9 — Démonstration : sortie + blocage de stock

3-4 min, LA PARTIE LA PLUS IMPORTANTE

⚠ **ATTENTION**

Le vertical slice actuel simplifie le circuit réel à 2 étapes (BonSortie avec un statut, puis exécution de la sortie), pas encore les 2 signatures séparées DSID/2e instance via la table Intervient — c'est prévu dans le modèle mais pas encore branché sur une vraie vue API. Dis-le clairement si le jury demande à voir ce circuit complet : "Le modèle de données le supporte déjà (table Intervient), la vue qui l'exploite pas encore."

🗣 **CE QUE JE DIS**

"Je crée un Bon de Sortie pour 999 cartouches, alors qu'il n'en reste que 45 en stock."

"*Raha mihoatra ny stock ny quantité demandée, dia tsy avela ny sortie.*"

👁 **CE QUE JE MONTRE**

Appel à `/api/sorties/effectuer/` avec ce Bon de Sortie → réponse en erreur 400, message explicite du dépassement.

🗣 **CE QUE JE DIS**

"Maintenant je refais la même chose avec une quantité raisonnable, 5 cartouches."

👁 **CE QUE JE MONTRE**

Réponse 201, la sortie est créée, avec la ligne de détail.

QUESTION DU JURY — "Comment empêchez-vous une sortie supérieure au stock ?"

RÉPONSE : La fonction `effectuer_sortie` dans services.py vérifie le stock disponible pour chaque ligne avant d'écrire quoi que ce soit en base ; si une seule ligne dépasse, toute l'opération est annulée grâce à une transaction atomique — jamais de sortie enregistrée à moitié.

Étape 10 — Traçabilité

2 min

🗣 CE QUE JE DIS

"Afaka fantarina hoe inona no nihetsika, oviana, ary iza no nampiditra an'ilay opération."

👁 CE QUE JE MONTRE

Ouvre `/api/mouvements/` dans la browsable API : montre la liste, avec le type de mouvement, la cible (article ou type+quantité), la date, l'utilisateur.

QUESTION DU JURY — "Comment gérez-vous les utilisateurs et leur suppression ?"

RÉPONSE : Un utilisateur n'est jamais supprimé, seulement désactivé (champ `actif`), pour ne jamais perdre la trace des opérations qu'il a enregistrées — cohérent avec la règle générale du projet : jamais de suppression définitive d'une donnée.

Étape 11 — Conclusion

2 min

🗣 CE QUE JE DIS

"Pour résumer : ce projet met en place une base de données complète et cohérente avec le métier réel de Paositra, une logique d'entrée qui distingue correctement matériel individualisé et consommable, et une règle de blocage de stock fonctionnelle et testée."

"Efa vita ny fototra ara-teknika — MCD, base de données, logique métier, API. Ny sisa dia ny interface React sy ny QR code, izay hovolavolaina amin'ny manaraka."

Déjà fonctionnel	Reste à développer
MCD complet en base PostgreSQL	Interface React
Logique individualisé / consommable	QR codes
Blocage de stock	Dashboard / avoirs par direction
Traçabilité (Mouvement)	Workflow complet à 2 signatures (autorisation/validation)
API REST + Admin	Tests automatisés, permissions par rôle détaillées

Questions du jury — réponses prêtes

Pourquoi Django ?
Framework mature, ORM intégré, écosystème DRF adapté à une API pour React.

Pourquoi PostgreSQL et pas SQLite ?
Contraintes d'intégrité robustes, adapté à un vrai système de gestion, pas un prototype.

Pourquoi DRF ?
Standard pour exposer une API REST propre depuis Django, avec sérialisation et permissions intégrées.

Pourquoi React ?
Séparation frontend/backend, interface dynamique, communication via API REST découplée du backend.

Pourquoi séparer Article et TypeArticle ?
TypeArticle décrit le modèle générique ; Article représente un exemplaire physique individuel qu'on doit pouvoir suivre précisément (référence, historique).

Pourquoi les consommables n'ont-ils pas d'Article ?
Parce que `gestion_individuelle=False` pour ces types : aucune valeur de traçabilité individuelle, juste une quantité globale à suivre.

Comment fonctionne la traçabilité ?
Chaque opération génère un Mouvement (entrée, sortie, retour...), lié soit à un Article, soit à un TypeArticle+quantité, avec l'utilisateur et la date.

Comment empêchez-vous une sortie supérieure au stock ?
Vérification du stock disponible avant toute écriture, dans une transaction atomique — soit tout passe, soit rien.

Comment gérez-vous les utilisateurs ?
Modèle personnalisé héritant d'AbstractUser Django, avec un rôle métier et un statut actif/inactif ; jamais de suppression, seulement désactivation.

Pourquoi avoir MOUVEMENT ?
Pour centraliser l'historique complet, sans avoir besoin d'une entité "Historique" séparée — tout se reconstruit via les Mouvement liés.

Pourquoi LIGNE_BSORTIE et LIGNE_SORTIE séparées ?
Une demande peut être refusée ou réduite ; on garde la trace de ce qui a été demandé séparément de ce qui a réellement été livré.

Comment avez-vous testé ?
Un script de vérification (test_scenarios.py) qui exécute les 3 scénarios métier (entrée individualisée, entrée consommable, sortie avec blocage) et vérifie les résultats attendus, plus des tests manuels de l'API via HTTP.

Qu'est-ce qui est réellement fonctionnel aujourd'hui ?
Le backend complet : modèles, logique métier d'entrée/sortie, blocage de stock, traçabilité, admin, API REST sur 7 endpoints.

Qu'est-ce qui reste à faire ?
Frontend React, QR codes, dashboard, workflow complet à 2 signatures via l'API, permissions par rôle détaillées, tests automatisés formels.

Quelles sont les limites actuelles ?
Le format de référence des articles est un espace réservé, pas encore le format réel Paositra (à confirmer) ; la sélection des articles "disponibles" pour une sortie ne tient pas encore compte des retours ; le workflow d'autorisation/validation à 2

signatures existe dans le modèle mais pas encore dans une vue API dédiée.

Comment le système pourrait-il évoluer ?

Ajout du frontend React connecté à l'API existante, génération de QR codes pointant vers la fiche article, dashboard de consultation des stocks et avoirs par direction, gestion complète des retours et corrections.

Checklist — 10 minutes avant la présentation

- ☐ Fichiers copiés chez moi : models.py, services.py, admin.py, serializers.py, views.py, urls.py, config/urls.py
- ☐ python manage.py makemigrations exécuté chez moi sans erreur
- ☐ python manage.py migrate exécuté chez moi sans erreur
- ☐ PostgreSQL démarré
- ☐ Superuser créé (python manage.py createsuperuser)
- ☐ python manage.py runserver démarre sans erreur
- ☐ Admin accessible et login fonctionne
- ☐ Scénario entrée individualisée (laptops) testé chez moi
- ☐ Scénario entrée consommable (cartouches) testé chez moi
- ☐ Scénario sortie bloquée (dépassement stock) testé chez moi
- ☐ Scénario sortie normale testé chez moi
- ☐ Traçabilité (liste des mouvements) vérifiée
- ☐ Fichiers à montrer ouverts dans l'éditeur (models.py, services.py, views.py)
- ☐ Terminal PowerShell ouvert, venv activé, dans backend/
- ☐ Ce guide relu une dernière fois

SI QUELQUE CHOSE CASSE PENDANT LA DÉMO

- **Si le serveur ne démarre pas :** montre directement le code (models.py, services.py) et le script test_scenarios.py déjà exécuté — explique la logique sur le papier/code plutôt qu'en live.
- **Si une requête API échoue :** reste calme, dis "je vais vérifier la logique dans le code" et ouvre services.py pour expliquer la règle sans dépendre du live.
- **Si PostgreSQL pose problème :** vérifie d'abord que le service est démarré (`Get-Service postgresql*` sous Windows) ; en dernier recours, montre le contenu de `test_scenarios.py` et son résultat déjà obtenu (à garder en capture d'écran, à préparer avant lundi).
- **Si tu bloques sur une question du jury :** il vaut mieux dire honnêtement "ce point n'est pas encore implémenté, mais voici comment je le modéliserais" plutôt que d'improviser une fausse certitude.